

Manual && Reproduction Report for

Watch and Act: Multi-orientation Open-set Scene Text Recognition via Dynamic Expert Routing

Hardware requirements

GPU:

Testing: NVIDIA GPU with Turing or later structure and more than 4 Gib of vram.

(yes pascals, maxwells or even fermis may work, but you gonna need to know what you are doing)

Training: 24GiB GPU with Turing or later structure

(In theory, 16GiB ones work too but I am not promising that.)

CPU: X86 cpus with AVX2 instruction set

(CPUs without AVX2 may or may not work with different versions of torch, I don't know)

RAM: 16GiB (Testing) 32Gib (Training)

Disk: 500GiB (well just dig out one old hdd from your bucket, or ebay and that's it)

Internet: Yes

Software Requirements

1. Fresh installed Archlinux with these packages:

`sudo vim plasma-meta nvidia openssh firefox dolphin konsole wget git less`

2. The user name is set to lasercat and it has to be a sudoer (or you may need to go through the code to replace paths if something went south).

3. ***Keep important data off this device!***

I don't want to wipe your data due to one or two failed cd commands followed by mv and/or rm

Trivia: Manul, aka the Pallas' cat, is the oldest cat species still alive on the earth.



Environment setup

1. install the following packages once you boot in.

```
sudo pacman -Syu pycharm-community-edition plasma-meta python-paramiko python-lmdb  
python-numba python-opencv python-pillow python-pip python-pyqtgraph python-pytorch-opt-  
cuda python-regex python-scikit-learn python-scipy python-torchvision-cuda python-tqdm python-  
xmldict make gcc cmake unzip python-setuptools kate firefox
```

2. make dirs

```
mkdir ~/cat ~/ssddata ~/hydra_saves ~/ssddata/anchors ~/Downloads
```

3. setup python stuff

```
mkdir ~/catenv/; python3 -m venv ${PWD}/catenv --system-site-packages;  
source ~/catenv/bin/activate; pip install easydict editdistance wandb  
cd ~/cat; git clone https://github.com/lancercat/make_envNG.git  
cd make_envNG/ ; sh pylcs.sh ;  
unzip pytorch_scatter-laser.zip ; cd pytorch_scatter-2.1.2/; python setup.py install
```

4. download the data

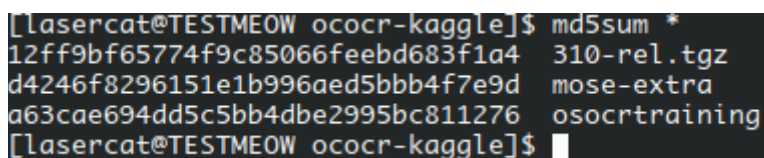
```
cd ~/Downloads;  
wget https://www.kaggle.com/api/v1/datasets/download/object300/mose-extra;  
wget https://www.kaggle.com/api/v1/datasets/download/vsdf2898kaggle/osocrtraining;
```

5. download models to ~/Downloads:

310-rel.tgz from <https://www.kaggle.com/models/object300/wna-zero>

6. check the md5sum:

```
cd ~/Download; md5sum *;
```



```
[laser@cat@TESTMEOW ococr-kaggle]$ md5sum *  
12ff9bf65774f9c85066feebd683f1a4 310-rel.tgz  
d4246f8296151e1b996aed5bbb4f7e9d mose-extra  
a63cae694dd5c5bb4dbe2995bc811276 osocrtraining  
[laser@cat@TESTMEOW ococr-kaggle]$
```

```
12ff9bf65774f9c85066feebd683f1a4 310-rel.tgz  
d4246f8296151e1b996aed5bbb4f7e9d mose-extra  
a63cae694dd5c5bb4dbe2995bc811276 osocrtraining
```

Note if you have different md5, it could be normal or not--- as we can have slightly corrupted archives as well --- they are pulled from kaggle too but at least ours work :)

we pulled them from kaggle to make sure the online version do work

In short, if you encounter problems AND have different md5sum --- try download again.

7. clone the code:

```
cd ~/cat; git clone https://github.com/lancercat/wna.git
```

8. run extract.sh:

```
cd ~/cat/wna/ ; sh extract.sh
```

⑨. stop and check:

```
[lancercat@TESTMEOW ~]$ ls ssddata/ hydra_saves/ cat
cat:
make_envNG wna

hydra_saves/:
aroute_nd_only-v6S-reg-nedmix-AAF-ohem01E_promelas_b32
aroute_nd_only-v6S-reg-nedmix-AAF-ohem01E_promelas_mjst_b32
aroute_nd_only-v6S-tiny-AAF
aroute_nd_only-v6S-tiny-AAF-01E
aroute_nd_only-v6S-tiny-AAF-01E-run2
aroute_nd_only-v6S-tiny-AAF-run2
aroute_nd_only-v6S-tiny-nedmix-AAF
aroute_nd_only-v6S-tiny-nedmix-AAF-ohem01
aroute_nd_only-v6S-tiny-nedmix-AAF-ohem01E
aroute_nd_only-v6S-tiny-nedmix-AAF-ohem01E-run2
aroute_nd_only-v6S-tiny-nedmix-AAF-run2
aroute_nd_only-v6S-tiny-nedmix-AAF-run2

pgroute_nd_only-v6S-tiny-AAF
pgroute_nd_only-v6S-tiny-AAF-01E
pgroute_nd_only-v6S-tiny-AAF-01E-run2
pgroute_nd_only-v6S-tiny-AAF-nedmix
pgroute_nd_only-v6S-tiny-AAF-nedmix-01E
pgroute_nd_only-v6S-tiny-AAF-nedmix-01E-run2
pgroute_nd_only-v6S-tiny-AAF-nedmix-run2
pgroute_nd_only-v6S-tiny-AAF-run2
rule_based-v6S-AAF
rule_based-v6S-AAF-ohem01E
rule_based-v6S-AAF-ohem01E-run2
rule_based-v6S-AAF-run2

ssddata/:
artdb_seen      ctwdb_seen      dicts      IC13_1015      lsvtdb_seen_NG  mltrchlat_seen_NG  mltrkr_hori      ST-abi
artdb_seen_NG   ctwdb_seen_NG   dictsv2     IIIT5k_3000    MJ-abi          mltrjp_hori        rctwtrdb_seen    SVT
athenaNG        CUTE80          IC03_867    lsvtdb_seen    mltrchlat_seen  mltrjp_hv         rctwtrdb_seen_NG

[lancercat@TESTMEOW ~]$
```



Reproducing ablative studies

1. run `ablative.py` (in a screen session as it takes some time):

```
python ablative.py 2>&1 | tee all.log
```

(our log is uploaded to github)

2. After the ablative experiments are finished (usually takes like a dozen hours on a GTX 1650), run `ablative2table.py` to compute mean and standard deviation:

```
/home/lasercat/catvenv/bin/python3.13 /home/lasercat/cat/wna/ablative2table.py
\newcommand{\LPAoBASEoJPNHVo6ZSLoLAoAVG}{40.97}
\newcommand{\LPAoBASEoJPNHVo6ZSLoLAoSTD}{1.02}
\newcommand{\LPAoW0ARoUeJPNHVo6ZSLoLAoAVG}{42.11}
\newcommand{\LPAoW0ARoUeJPNHVo6ZSLoLAoSTD}{0.51}
\newcommand{\LPAoPGoLoJPNHVo6ZSLoLAoAVG}{41.49}
\newcommand{\LPAoPGoLoJPNHVo6ZSLoLAoSTD}{1.01}
\newcommand{\LPAoPGoLoSoJPNHVo6ZSLoLAoAVG}{43.09}
\newcommand{\LPAoPGoLoSoJPNHVo6ZSLoLAoSTD}{0.33}
\newcommand{\LPAoPGoNoJPNHVo6ZSLoLAoAVG}{39.66}
\newcommand{\LPAoPGoNoJPNHVo6ZSLoLAoSTD}{0.36}
\newcommand{\LPAoPGoNoJPNHVo6ZSLoLAoSTD}{0.36}
\newcommand{\LPAoPGoNoJPNHVo6ZSLoLAoAVG}{43.17}
\newcommand{\LPAoPGoNoSoJPNHVo6ZSLoLAoSTD}{0.39}
\newcommand{\LPAoARoLoJPNHVo6ZSLoLAoAVG}{41.88}
\newcommand{\LPAoARoLoJPNHVo6ZSLoLAoSTD}{0.59}
\newcommand{\LPAoARoLoSoJPNHVo6ZSLoLAoAVG}{43.90}
\newcommand{\LPAoARoLoSoJPNHVo6ZSLoLAoSTD}{0.86}
\newcommand{\LPAoW0oHEMoJPNHVo6ZSLoLAoAVG}{42.05}
\newcommand{\LPAoW0oHEMoJPNHVo6ZSLoLAoSTD}{0.25}
\newcommand{\LPAoJPNHVo6ZSLoLAoAVG}{45.14}
\newcommand{\LPAoJPNHVo6ZSLoLAoSTD}{0.35}
\newcommand{\LPAoOHEMoJPNHVo6ZSLoLAoAVG}{44.53}
\newcommand{\LPAoOHEMoJPNHVo6ZSLoLAoSTD}{0.31}
-----
Process finished with exit code 0
```

3. save the output text as a txt file, say `reprod.txt`

(our log can be found in this github repo as `reprod.txt`)

4. diff them:

```
diff reprod.txt paperref.txt --color
```

Expect the results to be a bit different here and there due to float errors between different implementations of operators, which can be caused by different torch versions, gpu structures, vram, etc (yes, torch[1] and CUDNN[2] may choose different backend on different hardware)

[1] <https://discuss.pytorch.org/t/different-machines-different-results/100126>

[2] <https://docs.nvidia.com/deeplearning/cudnn/backend/latest/developer/core-concepts.html>

```
(catvenv) [lasercat@TESTMEOW wna]$ diff reprod.txt paperref.txt --color
2c2
< \newcommand{\LPAoBASEoJPNHVo6ZSLoLAoSTD}{1.02}
---
> \newcommand{\LPAoBASEoJPNHVo6ZSLoLAoSTD}{1.03}
7c7
< \newcommand{\LPAoPGoLoSoJPNHVo6ZSLoLAoAVG}{43.09}
---
> \newcommand{\LPAoPGoLoSoJPNHVo6ZSLoLAoAVG}{43.07}
9,10c9,10
< \newcommand{\LPAoPGoNoJPNHVo6ZSLoLAoAVG}{39.66}
< \newcommand{\LPAoPGoNoJPNHVo6ZSLoLAoSTD}{0.36}
---
> \newcommand{\LPAoPGoNoJPNHVo6ZSLoLAoAVG}{39.65}
> \newcommand{\LPAoPGoNoJPNHVo6ZSLoLAoSTD}{0.35}
13c13
< \newcommand{\LPAoARoLoJPNHVo6ZSLoLAoAVG}{41.88}
---
> \newcommand{\LPAoARoLoJPNHVo6ZSLoLAoAVG}{41.89}
18,20c18,20
< \newcommand{\LPAoW0oHEMoJPNHVo6ZSLoLAoSTD}{0.25}
< \newcommand{\LPAoJPNHVo6ZSLoLAoAVG}{45.14}
< \newcommand{\LPAoJPNHVo6ZSLoLAoSTD}{0.35}
---
> \newcommand{\LPAoW0oHEMoJPNHVo6ZSLoLAoSTD}{0.24}
> \newcommand{\LPAoJPNHVo6ZSLoLAoAVG}{45.15}
> \newcommand{\LPAoJPNHVo6ZSLoLAoSTD}{0.36}
22a23,24
```



run object310-rel/project310_v6SF_stability-re/aroute_nd_only-v6S-tiny-nedmix-AAF-ohem01E

Let's map them to the table

Note the performance differs a bit on this 1650, but not much (utilization btw)



Reproducing Large Model

Open-set

Run object310-rel/XL/aroute_nd_only-v6S-reg-nedmix-AAF-ohem01E_promelas_b32/osocr_benchall.py

```
/home/lasercat/cat/ana/ana_env/ana_framework_06/modules/concat_env_dev.py:67: UserWarning: To copy construct from a tensor, it is recommended to use sourceTensor.detach().clone() or sourceTensor.detach().clone().requires_grad_(True), rather than torch.tensor(sourceTensor).
  img=torch.stack([torch.tensor(i.dtype=this.get_type()) for i in imgList]).permute(0,3,1,2).contiguous().to(this.mean_data.device)-this.mean;
WARN: Corrupted image for 5871
Date: 2025-06-12 16:06:15.569554 ,TEST: KR-KR-GZSL ,Epoch: 0 ,Iter: 0 ,Total: 5170 ,ACR: 0.23965183752417796 ,Lenpred_ACR: 0.7253384912959381 ,FPS: 93.31770637168638
WARN: Corrupted image for 5875
Date: 2025-06-12 16:07:21.041607 ,TEST: JPNHV-JPNHV-GZSL ,Epoch: 0 ,Iter: 0 ,Total: 5074 ,ACR: 0.461568782026015 ,Lenpred_ACR: 0.8303113914071738 ,FPS: 91.47280815247775
WARN: Corrupted image for 5875
('Date': datetime.datetime(2025, 6, 12, 16, 0, 40, 4023), 'TEST': 'JPNHV-JPNHV-OSR', 'Epoch': 0, 'Iter': 0, 'Total': 5074.0, 'KACR': 0.7764070932922128, 'R': 0.7151178183743712, 'P': 0.954416961138742, 'F': 0.8176176782208696)
WARN: Corrupted image for 5875
('Date': datetime.datetime(2025, 6, 12, 16, 9, 11, 852653), 'TEST': 'JPNHV-JPNHV-GOSR', 'Epoch': 0, 'Iter': 0, 'Total': 5074.0, 'KACR': 0.7122658183103571, 'R': 0.6298440979955456, 'P': 0.8815461346633416, 'F': 0.7347362951415952)
WARN: Corrupted image for 5875
('Date': datetime.datetime(2025, 6, 12, 16, 9, 380266), 'TEST': 'JPNHV-JPNHV-OSTR', 'Epoch': 0, 'Iter': 0, 'Total': 5074.0, 'KACR': 0.738936256976452, 'R': 0.8050555342780544, 'P': 0.9239560439560439, 'F': 0.860417519433075)
Date: 2025-06-12 16:10:48.242145 ,TEST: JPN-JPN-GZSL ,Epoch: 0 ,Iter: 0 ,Total: 4809 ,ACR: 0.4804190571214767 ,Lenpred_ACR: 0.823896233474682 ,FPS: 98.4153275062363
('Date': datetime.datetime(2025, 6, 12, 16, 11, 28, 836917), 'TEST': 'JPN-JPN-OSR', 'Epoch': 0, 'Iter': 0, 'Total': 4809.0, 'KACR': 0.809720785938841, 'R': 0.7176199846507561, 'P': 0.9620978404858317, 'F': 0.8228674072679366)
('Date': datetime.datetime(2025, 6, 12, 16, 12, 9, 855949), 'TEST': 'JPN-JPN-GOSR', 'Epoch': 0, 'Iter': 0, 'Total': 4809.0, 'KACR': 0.763152227213496, 'R': 0.6254567981234568, 'P': 0.9076797385620915, 'F': 0.7391882908864937)
('Date': datetime.datetime(2025, 6, 12, 16, 12, 50, 407182), 'TEST': 'JPN-JPN-OSTR', 'Epoch': 0, 'Iter': 0, 'Total': 4809.0, 'KACR': 0.7335355648535565, 'R': 0.8063900810681927, 'P': 0.9489337822671156, 'F': 0.8718741943799947)
wandb:
wandb: You can sync this run to the cloud by running:
wandb: wandb sync /home/lasercat/hydra_logs/aroute_nd_only-v6S-reg-nedmix-AAF-ohem01E_promelas_b32/watch_and_control/aroute_nd_only-v6S-reg-nedmix-AAF-ohem01E_promelas_b32/wandb/offline-run-20250612_160516-78pek5ea
wandb: Find logs at: .../hydra_logs/aroute_nd_only-v6S-reg-nedmix-AAF-ohem01E_promelas_b32/watch_and_control/aroute_nd_only-v6S-reg-nedmix-AAF-ohem01E_promelas_b32/wandb/offline-run-20250612_160516-78pek5ea/logs
Process finished with exit code 0
```

Let's map them to the table

Split	Registered	Out-of-Set	Name	LA	Recall	Precision	FM
GZSL (Hori.)	Unique Kanji Shared Kanji, Kana, Latin,	∅	OSOCR-Large [4]	30.83	—	—	—
			OpenCCD-Large [23]	41.31	—	—	—
			OpenSAVR-XL [34]	42.58	—	—	—
			MOoSE-XL* [7]	39.56	—	—	—
			CFOR-XL [37]	44.47	—	—	—

Date: 2025-06-12 16:10:48.242145 ,TEST: JPN-JPN-GZSL ,Epoch: 0 ,Iter: 0 ,Total: 4809 ,ACR: 0.4804190571214767 ,Lenpred_ACR: 0.823896233474682 ,FPS: 98.4153275062363

			WnA-S-XL (Ours)*	48.02	—	—	—
OSTR (Hori.)	Shared Kanji, Unique Kanji	Kana Latin	OSOCR-Large [4]	58.57	24.46	93.78	38.80
			OpenSAVR-XL [34]	72.33	72.96	92.62	81.62
			MOoSE-XL* [7]	64.80	80.49	89.12	84.50
			CFOR [37]	71.80	86.36	89.52	87.91

'TEST': 'JPN-JPN-OSTR', 'Epoch': 0, 'Iter': 0, 'Total': 4809.0, 'KACR': 0.7735355648535565, 'R': 0.8063900810681927, 'P': 0.9489337822671156, 'F': 0.8718741943799947

			WnA-S-XL(Ours)*	77.35	80.68	94.89	87.21
--	--	--	-----------------	-------	-------	-------	-------

GZSL	Shared Kanji		MOoSE-XL	37.39	—	—	—
------	--------------	--	----------	-------	---	---	---

: 2025-06-12 16:07:21.041607 ,TEST: JPNHV-JPNHV-GZSL ,Epoch: 0 ,Iter: 0 ,Total: 5074 ,ACR: 0.461568782026015 ,Lenpred_ACR: 0.8303113914071738 ,FPS: 91.47280815247775

	Latin, Kana		WnA-S-XL(Ours)	46.14	—	—	—
--	-------------	--	----------------	-------	---	---	---

OSR	Shared Kanji	Unique Kanji	MOoSE-XL	75.56	74.05	95.79	83.53
-----	--------------	--------------	----------	-------	-------	-------	-------

ST': 'JPNHV-JPNHV-OSR', 'Epoch': 0, 'Iter': 0, 'Total': 5074.0, 'KACR': 0.7764070932922128, 'R': 0.7151178183743712, 'P': 0.954416961138742, 'F': 0.8176176782208696)

			WnA-S-XL(Ours)	77.64	71.45	95.44	81.72
--	--	--	----------------	-------	-------	-------	-------

GOSR	Shared Kanji	Kana	MOoSE-XL	59.81	63.83	81.89	71.74
------	--------------	------	----------	-------	-------	-------	-------

': 'JPNHV-JPNHV-GOSR', 'Epoch': 0, 'Iter': 0, 'Total': 5074.0, 'KACR': 0.7122658183103571, 'R': 0.6298440979955456, 'P': 0.8815461346633416, 'F': 0.7347362951415952)

	Latin		WnA-S-XL(Ours)	71.22	62.89	88.08	73.38
--	-------	--	----------------	-------	-------	-------	-------

OSTR	Shared Kanji	Latin	MOoSE-XL	61.75	80.01	88.18	83.90
------	--------------	-------	----------	-------	-------	-------	-------

'JPNHV-JPNHV-OSTR', 'Epoch': 0, 'Iter': 0, 'Total': 5074.0, 'KACR': 0.738936256976452, 'R': 0.8050555342780544, 'P': 0.9239560439560439, 'F': 0.860417519433075)

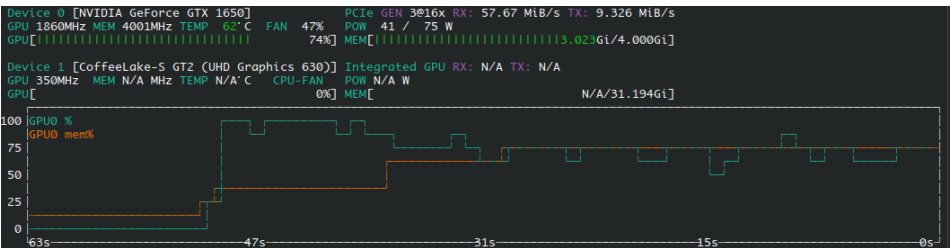
			WnA-S-XL(Ours)	73.89	80.50	92.39	86.04
--	--	--	----------------	-------	-------	-------	-------

GZSL	Hangul	∅	CFOR-XL [37]	22.14	—	—	—
------	--------	---	--------------	-------	---	---	---

TEST: KR-KR-GZSL ,Epoch: 0 ,Iter: 0 ,Total: 5170 ,ACR: 0.23965183752417796 ,Lenpred_ACR: 0.7253384912959381 ,FPS: 93.33770637168638

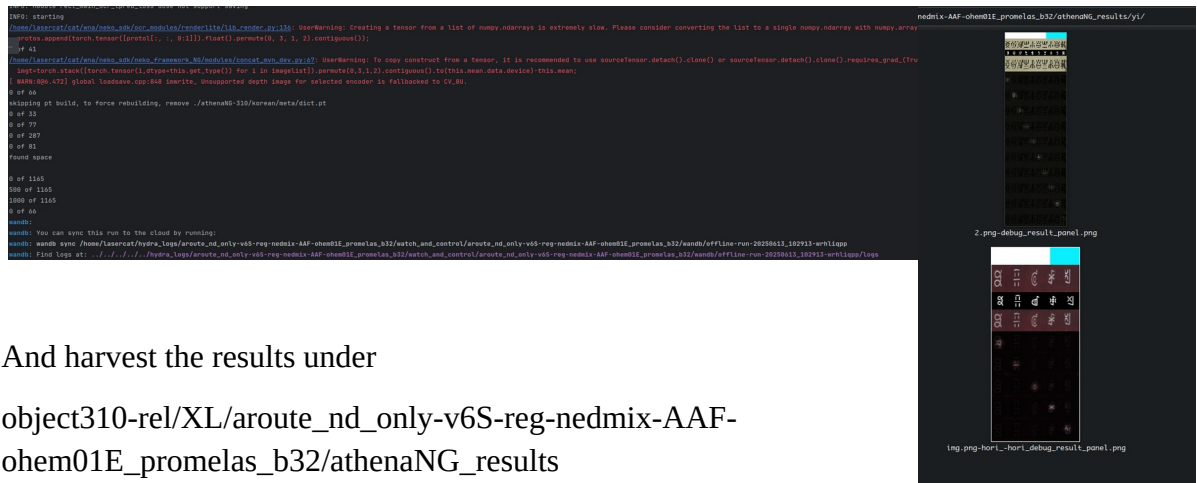
			WnA-S-XL (Ours)	24.00	—	—	—
--	--	--	-----------------	-------	---	---	---

Note the performance differs a bit on this 1650, but not much (utilization btw)



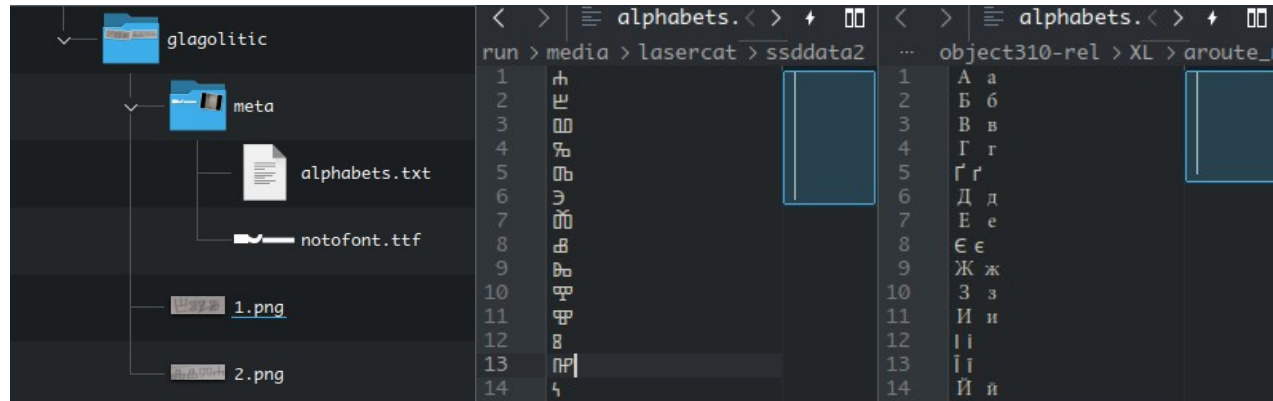
Reproducing Fig 1& Inference on your own data

Run object310-rel/XL/aroute_nd_only-v6S-reg-nedmix-AAF-ohem01E_promelas_b32/athena.py



And harvest the results under
object310-rel/XL/aroute_nd_only-v6S-reg-nedmix-AAF-ohem01E_promelas_b32/athenaNG_results

For inferencing your own data, you need to prepare the meta/alphabets.txt, the meta/notofont.ttf, and images.



The meta/alphabets.txt contains all alphabets, each a line. If an alphabet has more than one case, split them with space.

For the notofont.ttf, you search for the font here: <https://fonts.google.com/noto>
Download it, extract the regular variant, rename it to notofont.ttf, and add it to the folder.

Close-set

Run object310-rel/XL/aroute_nd_only-v6S-reg-nedmix-AAF-ohem01E_promelas_mjst_b32/test.py

Date: 2025-06-17 13:14:36.333460 ,TEST: IIIT5k-close ,Epoch: 0 ,Iter: 0 ,Total: 3000 ,ACR: 0.9036666666666666 ,Lenpred ACR: 0.8713333333333333 ,FPS: 97.06534608531325

Table 3. Accuracy and speed comparisons on the IIIT5k dataset [38]. The second reference indicates the source of (re)implementation and speed test.

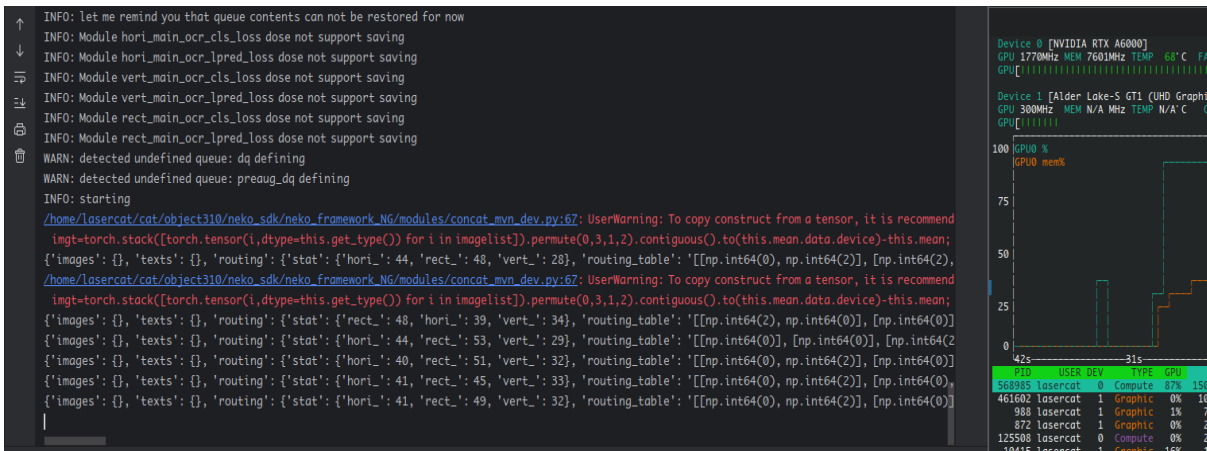
Method	Venue	Line Accuracy	Testing GPU	TFlops	FPS
Parseq [39][40]	ECCV'22	97.0	1080Ti	12	84.7
CPPD [40]	Arxiv'23	97.7	1080Ti	12	206
CRNN[41][40]	CVPR'17	82.9	1080Ti	12	159
ViTSTR[42]	ICDAR'21	88.4	2080Ti	13.6	102
WnA-S-XL	-	90.37	Tesla P40	12	327.3



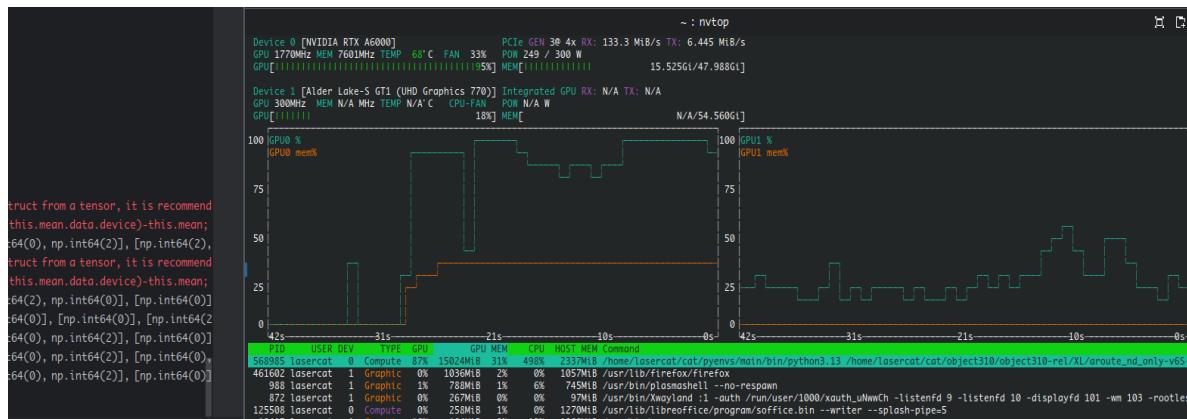
Training from Scratch

Run `train.py` under the method folder you choose. For example to train the large model:

object310-rel/XL/aroute_nd_only-v6S-reg-nedmix-AAF-ohem01E_promelas_b32/train.py



And watch it cook. Utilization BTW



If you want to train with your own data, please try to follow the instructions from the vanilla osocr-data repo, the lmdbs are fully compatible.

The newly added `dictsv2/lang/vismeta.pt` builders can be reverse engineered from the athena scripts. Writing this manual takes too long already so I am not going to write an extensive document for now... pls understand the coding is developed and maintained mostly by a single soul (me).

I may come back if this thing gets a journal extension...

GLHF.



That's it

If anything is buggy, doesn't work, or you just want to chat, open an issue or contact us

Chang: lasercat@gmx.us (all time); chang.liu@ltu.se (work time please)

Elisa: elisa.barney@ltu.se

BTW... Since “manul” is spelled like “manual”, so you will see a manul at the end of each section as an end-of-section icon.

See ya && GLHF

